CS6886: Project Proposal

EXPLORING LLS: SCALING, EARLY-EXIT AND ADAPTIVE-DEPTH LOCAL LEARNING

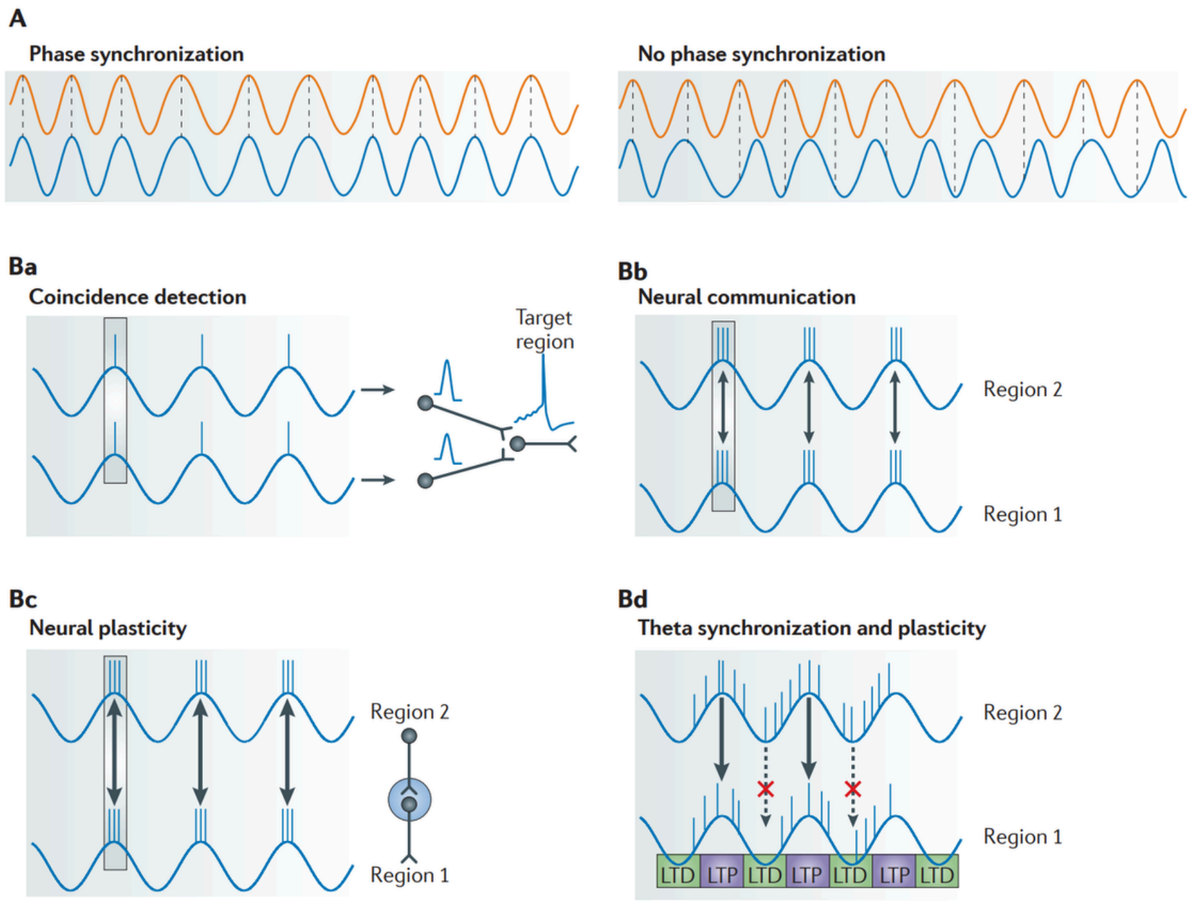
HAYAGRIV VENKATESH ED23B024
YUKASH RAM CS23B069



Start

WHAT IS LLS?

Local Learning Rule inspired by neural synchronization activity that happens in our brain.



Source: The role of phase synchronization in memory processes

WHY LLS?

WHAT ABOUT OTHER LOCAL LEARNING RULES?

- **Feedback Alignment / DFA: fixed random feedback matrices, but BP-like memory O(Ln); poor scaling to deep networks.**
- **DRTP: fixed random targets; O(max(n)) memory, but lower accuracy (68.96% on CIFAR10)**
- **Forward-Forward / PEPITA - Slow Convergence**
- **Soft-winner takes it all and other Hybrid mode - Scalability vs Efficiency trade-off**
- **LLS - Computationally light functions, Less memory.**

Method	Time	Memory	Model	MNIST	CIFAR10	CIFAR100
BP (baseline)	$O(Ln^2)$	$O(Ln)$	SmallConvL	99.62 ± 0.020	87.57 ± 0.13	62.25 ± 0.29
DFA	$O(LCn)$	$O(Ln)$	SmallConvL	97.90 ± 0.17	71.53 ± 0.38	44.93 ± 0.52
			[28]	98.98 ± 0.02	73.10 ± 0.50	41.00 ± 0.30
DRTP	$O(LCn)$	$O(n_{max})$	[9]	98.52 ± 0.15	68.96 ± 0.45	–
PEPITA	$O(Ln^2)$	$O(n_{max})$	[7]	98.29 ± 0.13	56.33 ± 1.35	27.56 ± 0.60
$\text{LLS}_{\text{square}}$ (Ours)	$O(LCn)$	$O(n_{max})$	SmallConvL	99.57 ± 0.03	84.10 ± 0.27	55.32 ± 0.38

Source: Local Learning Rule for Deep Neural Networks Inspired by Neural Activity Synchronization

HOW LLS?

Each layer projects its own output onto a fixed, untrained basis generated from a periodic function.

Traditional BP:

O(Ln) memory , O(Ln*n) computation

LLS:

O(max(n)) memory , O(Ln*C) computation

$$\nabla_{\mathbf{w}^{(l)}}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l)}} \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{w}^{(l)}}$$

$$= \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \prod_{i=l+1}^{L} \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(i-1)}} \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{z}^{(l)}} \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{w}^{(l)}}$$

1. $\mathbf{h}^{(l)} = f(\mathbf{z}^{(l)})$
2. $\mathbf{h}^{(l)}\mathbf{b}^{(l)\top}$ (Projection on the fixed basis)
3. $\mathbf{p}^{(l)} = \text{softmax}(\mathbf{h}^{(l)}\mathbf{b}^{(l)\top})$
   $\mathcal{L}_{\text{CE}}^{(l)} = \mathbf{y}^{*}\log(\mathbf{p}^{(l)})$
4. $\mathbf{e}^{(l)} = \mathbf{p}^{(l)} - \mathbf{y}^{*}$
5. $\mathbf{e}^{(l)}\mathbf{b}^{(l)}$ (Learning signal for layer l)

$$\Delta \mathbf{w}^{(l)} = \left(\mathbf{e}^{(l)}\mathbf{b}^{(l)} \odot f'(\mathbf{z}^{(l)})\right)^{\top} \mathbf{h}^{(l-1)}$$

$$\mathbf{w}^{(l)} := \mathbf{w}^{(l)} - \eta \Delta \mathbf{w}^{(l)} \quad \text{(Weight update)}$$

Source: Local Learning Rule for Deep Neural Networks Inspired by Neural Activity Synchronization

OPEN PROBLEMS WITH LLS RULE

SCALABILITY

LLS has been experimented with only smaller models such as MobileNet and VGG8. It is unclear how well LLS Scales and there is a inherent bottleneck on number of classes it can classify accurately.

ATTENTION , INTERPRETABILITY

It is not very clear whether LLS and Attention mechanism work together. LLS, by design, gives us outputs per layer - could they be interpreted in a way that will provide us efficiency in memory/compute?

LOCAL LEARNING DEPTH

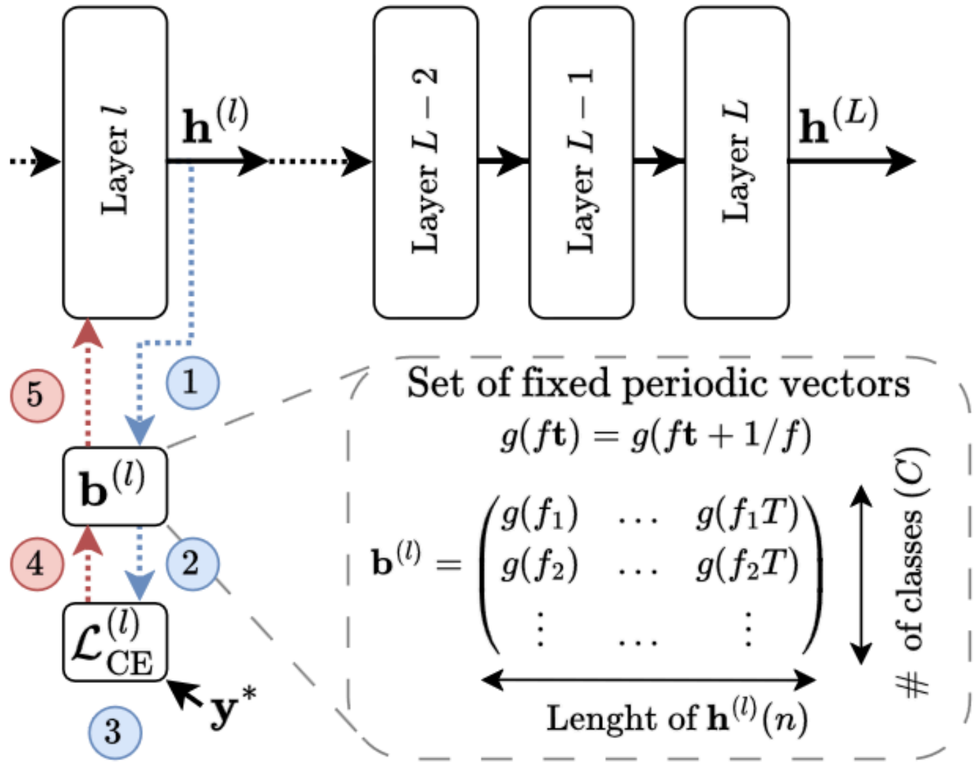
LLS has been experimented with each layer performing local loss computation. It is not known whether LLS will pair well with Backpropagation with lesser granularity.

Method	Model	Accuracy (mean±std)	# MAC[1] ($\times 10^6$)	Memory[1] (MB)	Additional params
CIFAR10					
BP	VGG8	94.12 ± 0.12	719.33	1082	-
Local Losses	VGG8	91.93 ± 0.07	2.56	576	1.02×10^5
LLS_{square} (Ours)	VGG8	88.64 ± 0.12	2.46	574	0
LLS-M_{square} (Ours)	VGG8	90.43 ± 0.24	2.46	574	70
LLS-MxM$_{square}$ (Ours)	VGG8	90.89 ± 0.09	2.46	574	700
CIFAR100					
BP	VGG8	73.69 ± 0.39	719.40	1083	-
Local Losses	VGG8	69.26 ± 0.36	5.33	598	1.02×10^6
LLS_{square} (Ours)	VGG8	58.84 ± 0.33	4.30	577	0
LLS-M_{square} (Ours)	VGG8	62.55 ± 0.24	4.31	577	700
LLS-MxM$_{square}$ (Ours)	VGG8	68.81 ± 0.19	4.51	578	0.70×10^5

Source: Local Learning Rule for Deep Neural Networks Inspired by Neural Activity Synchronization

OUR IDEAS AND APPROACHES – EARLY EXIT

- Inherently, most datasets are skewed with “easier” inputs and “harder” inputs
- LLS gives us “output” every layer
- Can we decide based on layer-wise outputs? What are the signals?
- Do “Easier” images sync sooner? which classes exit early?



OUR IDEAS AND APPROACHES – GRANULARITY

- Do we need LLS every layer, if at all we are using LLS?
- What about every 2 layers or 3 layers?
- How about we infer the layer granularity by some other way?

OUR IDEAS AND APPROACHES – SCALABILITY

- LLS has been tested only in smaller models such as VGG8 , MobileNet.
- Inferring Early Exit will be extremely hard. Even then, it wouldn’t save much if the model is not deep enough.
- What about LLS on ResNet? How does one handle skip connections?

OUR IDEAS AND APPROACHES – ATTENTION

- From the LLS paper, it is clear that LLS is bottlenecked by number of classes in the dataset.
- What about Vision Transformers? Does Attention work well with LLS?
- What about having different architecture for Language Models with LLS? (Multiple classifier heads)

IMPLEMENTATION PLAN

Baseline:

Reproduce LLS VGG8 CIFAR10 (88.64%) and MobileNet VWW (81.91%) with the official repository

Granularity:

1. Uniform: k = 1, 2, 3
2. Resolution-aligned: boundaries at pooling layers.
3. Budget-driven: best partition under a fixed training-memory budget
4. Search (non-local reference) based on SEDONA

Early Exit:

1. Diagnostics: per-layer accuracy, MAC profile, overthinking rate, basis orthogonality.
2. Exit policies: confidence, patience, sync ratio, hybrids; thresholds tuned on validation only
3. BP + frozen-backbone internal classifiers and BP + jointly trained exits.
4. Local Losses + exits
5. Analysis: correlation of exit depth with prediction depth; per-class exit histograms

Scalability:

1. Apply LLS to ResNet
2. LLS to ViT-Tiny
3. Multiple-classifiers

DELIVERABLES

At the end of this project, we aim to provide answers to the following primary questions:

- Does LLS provide us free signals/interpretation that allows us to perform early exit?
- Can LLS be scaled to ResNet and other larger models? If no, what is the bottleneck?
- Can the number of layers that use LLS be varied? If no, why?
- Do LLS and Attention go hand-in hand? If no, why? (Exploration)

Apart from this, we also promise to provide

- Experiments and results for
  - Early Exit Policies: Softmax confidence , Patience , Sync-ratio
  - Early Exit Heads: Fixed Basis , Linear exit , BP + Convolutional Exit
  - Granularity: Uniform , Boundaries at pooling layers , SEDONA search.
  - Scalability: ResNet , ViT(exploration)
- Implementation and code for all the above
- Report containing our insights, hypothesis, graphs , observations and further ideas

RESULTS REPRODUCED

Function $g(.)$	Model	CIFAR10
Cosine	SmallConv	75.82 ± 0.39
Square		77.79 ± 0.31
Random		74.19 ± 0.57
Cosine	VGG8	86.92 ± 0.27
Square		88.64 ± 0.12
Random		90.45 ± 0.09

Table 1. LLS's performance comparison with different function

model	mode	dataset	final	best	best epoch	paper mean	paper std
SmallConv	cosine	CIFAR10	71.42	75.91	10	75.82	0.39
SmallConv	square	CIFAR10	74.01	78.35	17	77.79	0.31
SmallConv	random	CIFAR10	78.76	79.98	22	74.19	0.57
VGG8	cosine	CIFAR10	84.39	85.61	98	86.92	0.27
VGG8	square	CIFAR10	86.25	87.18	52	88.64	0.12
VGG8	random	CIFAR10	89.91	89.91	167	90.45	0.09

Method	Time	Memory	Model	CIFAR10
BP (baseline)	$O(Ln^2)$	$O(Ln)$	SmallConvL	87.57 ± 0.13
DFA	$O(LCn)$	$O(Ln)$	SmallConvL	71.53 ± 0.38
			[28]	73.10 ± 0.50
DRTP	$O(LCn)$	$O(n_{max})$	[9]	68.96 ± 0.45
PEPITA	$O(Ln^2)$	$O(n_{max})$	[7]	56.33 ± 1.35
$\text{LLS}_{\text{square}}$ (Ours)	$O(LCn)$	$O(n_{max})$	SmallConvL	84.10 ± 0.27

Table 2. Comparison with local learning algorithms

model	mode	dataset	final	best	best epoch	paper mean	paper std
SmallConvL	BP	CIFAR10	87.27	87.54	82	87.57	0.13
SmallConvL	DFA	CIFAR10	76.27	77.09	28	71.53	0.38
SmallConvL	LLS-square	CIFAR10	80.26	83.84	16	84.1	0.27

PROJECT ROADMAP AND TENTATIVE TIMELINE

INITIAL PHASE	WEEKS 1-3 (YUKASH)	WEEKS 3-5 (YUKASH AND HAYAGRIV)	WEEKS 1-3 (HAYAGRIV)	EXPLORATORY (YUKASH AND HAYAGRIV)
LITERATURE SURVEY AND BASELINE	LLS EARLY-EXIT IMPLEMENTATION	LLS ADAPTIVE DEPTH LOCAL LEARNING	LLS SCALABILITY:ResNet	LLS AND ATTENTION: ViT
Conduct Extensive Literature survey on existing techniques, theoretical foundations for the techniques used and establish baseline statistics by reproducing existing results.	Ideate on Early-exit methods and implement the optimization. Test and Benchmark the results and compare it with the established baseline.	Ideate on Granularity of the local layer depth. Implement variable depth-gating (fixed and learnable). Test and Benchmark the results and compare it with the baseline.	Ideate on ways to make LLS rules compatible with ResNet Architecture. Implement LLS rule on ResNet and compare the results with other Backprop-based results.	Find out whether Attention and LLS go hand-in-hand. Come up with a way to apply LLS with Attention Mechanism. Compare the results with existing Backprop-based results.

REFERENCES

Local Learning Rule:

- Apolinario, M. P. E., Roy, A., & Roy, K. "LLS: Local Learning Rule for Deep Neural Networks Inspired by Neural Activity Synchronization."

Early Exit:

- Panda, P., Sengupta, A., & Roy, K. "Conditional Deep Learning for Energy-Efficient and Enhanced Pattern Recognition.”
- Teerapittayanon, S., McDanel, B., & Kung, H. T. "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks."
- Kaya, Y., Hong, S., & Dumitras, T. "Shallow-Deep Networks: Understanding and Mitigating Network Overthinking."

Granularity:

- Belilovsky, E., Eickenberg, M., & Oyallon, E. "Greedy Layerwise Learning Can Scale to ImageNet."
- Belilovsky, E., Eickenberg, M., & Oyallon, E. "Decoupled Greedy Learning of CNNs." ICML 2020.
- Pyeon, M., Moon, J., Hahn, T., & Kim, G. "SEDONA: Search for Decoupled Neural Networks toward Greedy Block-wise Learning."
- “Towards Interpretable Deep Local Learning with Successive Gradient Reconciliation." (2024).


ANY
QUESTIONS?